PROJECT LAB REPORT

Title

Lucky-Mart Shopping Management System Using C++

Student Name : Nimra Arshad

University : Riphah International University Islamabad I-14

Semester : 4rth

1. Objective

To design and implement a flexible and modular console-based shopping management system in C++ that effectively demonstrates the fundamental principles of Object-Oriented Programming (OOP), namely Encapsulation, Inheritance, Polymorphism, and Abstraction.

2. Introduction

The Lucky-Mart Shopping Management System is a command-line application developed entirely in C++. This project serves as a practical implementation of OOP concepts to model a real-world shopping scenario. The system provides a comprehensive user interface for product browsing, dynamic management of a shopping cart using a Linked List, calculation of the final bill, and a promotional lucky draw feature. The architecture is designed to be highly maintainable and easily extensible for adding new product types or functionalities.

3. System Features

Menu-Driven Interface: Allows users to easily navigate between primary functions (Show Products, Add to Cart, View Cart, Lucky Draw, Exit).

Hierarchical Product Catalog: Products are classified into specialized categories (Electronics, Clothing, Grocery), each holding category-specific data (e.g., warranty, size, expiry).

Dynamic Cart Management: The Cart class utilizes a Linked List to dynamically store products, allowing for efficient addition of items without pre-defining the cart size.

Total Bill Calculation: The viewCart() function calculates and displays the aggregated price of all items currently in the cart.

Polymorphic Display: A single display() call correctly shows unique details (like size or expiry) for different product types.

4. OOP Concepts Implementation (The Four Pillars)

The core strength of the Lucky-Mart system lies in the strategic application of the four pillars of OOP:

Inheritance :

Mechanism: The Product class acts as the Base Class. The Electronics, Clothing, and Grocery classes are Derived Classes that inherit common attributes (ID, Name, Price) and methods from Product.

Code Example:

class Electronics : public Product { ... };

This ensures code reusability, as the specialized classes do not need to redefine basic product properties.

Encapsulation :

Mechanism: Data integrity is maintained by declaring all attributes (id, name, price, warranty, etc.) as private. Access to this data is only permitted via controlled public methods, such as the constructor or the Getter methods (getPrice() and getId()).

Code Example:

private: int id; string name; in Product class.

public: double getPrice() { return price; } is the controlled access point.

Polymorphism (Many Forms):

Mechanism: Polymorphism is achieved through the use of Virtual Functions and Base Class Pointers. The Product class defines virtual void display().

Code Example:

The Cart class stores items using Product* pointers. When t->data->display() is called inside viewCart(), the C++ runtime determines which specific display() function (i.e., Electronics::display(), Clothing::display(), or Product::display()) to execute, based on the object's actual type.

Abstraction :

Mechanism: Abstraction is implemented by defining the public interface that users (or other classes) interact with, while hiding the complex internal implementation.

Code Example:

The user only calls cart.addItem(product_ptr) or s.showMenu(). The complex details of how the cart manages the Linked List (struct Node) and how memory is allocated (new Node()) are hidden within the Cart class, providing a clean, abstract interface.

5. Conclusion :

The Lucky-Mart Shopping Management System is a comprehensive and functional project that successfully achieves its objective of modeling a business scenario using Object-Oriented Programming in C++.

By systematically applying Inheritance, we established a flexible product hierarchy. Encapsulation ensures data security and integrity across all classes. Most importantly, Polymorphism enabled the system to handle varied product types seamlessly through a single interface call (display), making the

code highly dynamic and scalable. The use of a Linked List for the shopping cart further demonstrates proficiency in managing dynamic data structures alongside complex OOP architecture. This project serves as a strong foundation for developing more extensive inventory and transaction management systems.

6. Source Code :

main.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;
class Product {
private:
    int id;
    string name;
    double price;
public:
    Product(int idd, string namee, double pricee) {
        id = idd;
        name = namee;        price = pricee;  }
    virtual ~Product() {}
    virtual void display() {
        cout << "🆔 Product ID : " << id << endl;
        cout << "📦 Name       : " << name << endl;
        cout << "💰 Price      : " << price << " PKR" << endl;    }
    double getPrice() { return price; }
    int getId() { return id; }};
class Electronics : public Product {
private:
    int warranty;
public:
    Electronics(int idd, string namee, double pricee, int warrantyy)
        : Product(idd, namee, pricee) {
        warranty = warrantyy;    }
    void display() {
        cout << "🔌 Electronics Item\n";
        Product::display();
        cout << "🛡 Warranty   : " << warranty << " year(s)\n";  }};
class Clothing : public Product {
```

main.cpp

```cpp
class Clothing : public Product {
private:
    string size;
public:
    Clothing(int idd, string namee, double pricee, string sizee)
        : Product(idd, namee, pricee) {
        size = sizee;}
    void display() {
        cout << "👕 Clothing Item\n";
        Product::display();
        cout << "📏 Size       : " << size << endl; }};
class Grocery : public Product {
private:
    string expiry;
public:
    Grocery(int idd, string namee, double pricee, string expiryy)
        : Product(idd, namee, pricee) {
        expiry = expiryy;    }
    void display() {
        cout << "🛒 Grocery Item\n";
        Product::display();
        cout << "📅 Expiry     : " << expiry << endl;    }};
class Cart {
private:
    struct Node {
        Product* data;
        Node* next;    };
    Node* head;
public:
    Cart() { head = NULL; }
```

main.cpp

```cpp
    ~Cart() {
        while (head != NULL) {
            Node* temp = head;
            head = head->next;
            delete temp;     }    }
    void addItem(Product* p) {
        Node* n = new Node();
        n->data = p;
        n->next = NULL;
        if (head == NULL) head = n;
        else {
            Node* t = head;
            while (t->next != NULL) t = t->next;
            t->next = n;        }
        cout << "✅ Item added to cart\n";    }
    void viewCart() {
        if (head == NULL) {
            cout << "🛒 Cart is empty\n";
            return; }
        Node* t = head;
        double total = 0;
        int i = 1;
        cout << "\n📋 YOUR CART\n";
        while (t != NULL) {
            cout << "\nItem " << i++ << endl;
            t->data->display();
            cout << "⏳ Status : Pending\n";
            total += t->data->getPrice();
            t = t->next; }
        cout << "\n💰 Total : " << total << " PKR\n";    }};
class Shop {
```

main.cpp

```cpp
class Shop {
private:
    struct PNode {
        Product* data;
        PNode* next;
    };
    PNode* productHead;
    Cart cart;
public:
    Shop() { productHead = NULL; }
    ~Shop() {
        while (productHead != NULL) {
            PNode* temp = productHead;
            productHead = productHead->next;
            delete temp->data;
            delete temp;    }   }
    void addProduct(Product* p) {
        PNode* n = new PNode();
        n->data = p;
        n->next = NULL;
        if (productHead == NULL) productHead = n;
        else {
            PNode* t = productHead;
            while (t->next != NULL) t = t->next;
            t->next = n;   }  }
void loadProducts() {
    addProduct(new Electronics(1, "Laptop", 50000, 1));
    addProduct(new Electronics(2, "Headphones", 2500, 1));
```

main.cpp

```cpp
    addProduct(new Grocery(5, "Chocolate Box", 1500, "2026-06-30"));
    addProduct(new Grocery(6, "Cooking Oil (1L)", 2200, "2026-12-15"));}
    void showProducts() {
        PNode* t = productHead;
        cout << "\n📦 AVAILABLE PRODUCTS\n";
        while (t != NULL) {
            t->data->display();
            cout << endl;
            t = t->next;      }    }
    void addToCart() {
        int id;
        cout << "Enter Product ID: ";
        cin >> id;
        PNode* t = productHead;
        while (t != NULL) {
            if (t->data->getId() == id) {
                cart.addItem(t->data);
                return;     }
            t = t->next;}
        cout << "❌ Product not found\n";
    }
    void luckyDraw() {
        char userChar;
        char luckyChar = 'R';
        cout << "\n🎁 LUCKY DRAW 🎁\n";
        cout << "Choose one letter (A-Z): ";
        cin >> userChar;
        if (toupper(userChar) == luckyChar) {
            cout << "\n🎉 CONGRATULATIONS!\n";
            cout << "You won a FREE PRODUCT 🎁\n";
            cout << "📦 Chocolate Box added for FREE!\n";
```

```
            cout << "You won a FREE PRODUCT 🎁\n";
            cout << "🍫 Chocolate Box added for FREE!\n";
            cart.addItem(new Grocery(99, "FREE Chocolate Box", 0, "2026-12-31"));
        } else {
            cout << "\n❌ Better luck next time 😊\n";        }   }
    void showMenu() {
        int choice;
        do {
            cout << "\n✨ Welcome to Lucky-Mart✨\n";
            cout << "  1. Show Products\n";
            cout << "  2. Add to Cart\n";
            cout << "  3. View Cart\n";
            cout << "  4. Lucky Draw 🎁\n";
            cout << "  5. Exit\n";
            cout << "  Enter choice: ";
            cin >> choice;
            if (choice == 1) showProducts();
            else if (choice == 2) addToCart();
            else if (choice == 3) cart.viewCart();
            else if (choice == 4) luckyDraw();
        } while (choice != 5);
        cout << "🙏 Thank you for shopping\n";    }};
int main() {
    Shop s;
    s.loadProducts();
    s.showMenu();    return 0;}
```

- **<u>OUTPUT</u> :**

```
Output

✨ Welcome to Lucky-Mart✨
  1. Show Products
  2. Add to Cart
  3. View Cart
  4. Lucky Draw 🎁
  5. Exit
  Enter choice: 1

📦 AVAILABLE PRODUCTS
🔌 Electronics Item
🆔 Product ID : 1
📦 Name       : Laptop
💰 Price      : 50000 PKR
🛡 Warranty   : 1 year(s)

🔌 Electronics Item
🆔 Product ID : 2
📦 Name       : Headphones
💰 Price      : 2500 PKR
🛡 Warranty   : 1 year(s)

👕 Clothing Item
🆔 Product ID : 3
📦 Name       : T-Shirt
💰 Price      : 1200 PKR
📏 Size       : M

👕 Clothing Item
🆔 Product ID : 4
📦 Name       : Jeans
💰 Price      : 2000 PKR
```

```
Output

🛒 Grocery Item
🆔 Product ID : 5
📦 Name       : Chocolate Box
💰 Price      : 1500 PKR
📅 Expiry     : 2026-06-30

🛒 Grocery Item
🆔 Product ID : 6
📦 Name       : Cooking Oil (1L)
💰 Price      : 2200 PKR
📅 Expiry     : 2026-12-15

✨ Welcome to Lucky-Mart✨
  1. Show Products
  2. Add to Cart
  3. View Cart
  4. Lucky Draw 🎁
  5. Exit
  Enter choice:
2
Enter Product ID: 3
✅ Item added to cart

✨ Welcome to Lucky-Mart✨
  1. Show Products
  2. Add to Cart
  3. View Cart
  4. Lucky Draw 🎁
  5. Exit
  Enter choice:
```

```
Output

  Enter choice:
2
Enter Product ID: 3
✅ Item added to cart

✨ Welcome to Lucky-Mart✨
  1. Show Products
  2. Add to Cart
  3. View Cart
  4. Lucky Draw 🎁
  5. Exit
  Enter choice: 4

🎁 LUCKY DRAW 🎁
Choose one letter (A-Z): R

🎉 CONGRATULATIONS!
You won a FREE PRODUCT 🎁
📦 Chocolate Box added for FREE!
✅ Item added to cart

✨ Welcome to Lucky-Mart✨
  1. Show Products
  2. Add to Cart
  3. View Cart
  4. Lucky Draw 🎁
  5. Exit
  Enter choice: 5
🙏 Thank you for shopping
```